

LoanInNeed - Project Handover Document

Date: March 16, 2026 **Project:** LoanInNeed Digital Lending Platform

1. Project Overview

LoanInNeed is a digital lending platform featuring a modern microservices-styled decoupled architecture. The platform supports loan origination, multi-step KYC and document verification, as well as a robust attribution and referral system for business partners (DSA, BC, Affiliates).

2. Architecture & Technology Stack

Frontend (Directory: `\LIN_Front`)

- **Framework:** Next.js 15 (App Router) with React 19.
- **Language:** TypeScript.
- **Styling:** Tailwind CSS v4, Radix UI components, Framer Motion.
- **State/Forms:** `@tanstack/react-form`, `react-hook-form`, `zod` for validation schemas.
- **CMS Integration:** Sanity.io (`/sanity`).
- **API Interceptor:** Custom `ApiClient` (`lib/api.ts`) managing JWT token injection, FormData boundary setting, and standardized error normalization based on backend responses.

Backend (Directory: `\Backend`)

- **Framework:** Node.js with Express 5.0.
 - **Language:** JavaScript.
 - **Database ORM:** Prisma ORM.
 - **Database:** PostgreSQL (currently hosted on Render).
 - **Storage:** Supabase (S3 compatible) configured via `multer`.
 - **Core Security:** `helmet`, `cors` (Whitelisting Vercel and local hosts), HMAC-SHA256 based cryptographic signature for the referral/partner system.
 - **Background Jobs:** Dedicated worker script (`scripts/losWorker.js`) deployed alongside the main server for Loan Origination System (LOS) API syncs.
-

3. Deployment Infrastructure

The system is currently deployed on free/dev tiers for demonstration and testing.

- **Frontend:** Vercel (<https://loaninneed.vercel.app>)
- **Backend:** Railway (<https://loaninneed-backend-production-7028.up.railway.app>)
- **Database:** Render (PostgreSQL)
- **Storage:** Supabase S3 bucket

Critical Setup Variables:

- Frontend uses `NEXT_PUBLIC_BACKEND_URL=https://loaninneed-backend-production-7028.up.railway.app`
 - Backend maps `CORS` correctly to the Frontend domain.
-

4. Key Application Workflows

4.1. End-User Signup & Verification

The user registration process follows a 6-step flow (`hooks/useSignup.ts`):

1. **OTP Verification:** via `/api/auth/phone/request-otp` & `verify-otp`.
2. **Profile Completion:** Posts name, DOB, auto-generated email.
3. **KYC Submission:** Validates details like job stability and income.
4. **Document Uploads:** Submits salary slips and bank statements (`/api/document/submit`). File size and format checking is enforced.
5. **Aadhaar Verification:** Uses OTP or bypass logic for KYC verification.
6. **Location & Selfie:** Captures lat/long (`/api/users/location`) and selfie (`/api/selfie/upload`).

4.2. Partner Attribution Workflow

- Partners (Affiliates, DSA, BC) sign up and receive a signed referral link containing `pid` (Partner ID), `ts` (Timestamp), and an HMAC `sig` (Signature).
 - When a user clicks a referral link, the frontend `ReferralTracker.tsx` validates and persists attribution data in `localStorage` (`lin_attribution`).
 - Upon user registration, this data is passed to the backend, validated, and linked to the partner account. Commissions and tracked loans are surfaced at `/api/partners/earnings`.
-

5. Mock Data & Known Bypasses

Due to the lack of active third-party integrations for messaging and verification, the platform uses specific bypasses for QA testing:

- **Mobile OTP:** The backend does not send SMS. ALWAYS use the Master OTP `261102` for phone verifications.
 - **SMTP Emails:** Registration emails are currently not dispatched. Partner configuration responses provide credentials on the frontend instantly.
 - **PAN / Aadhaar:** No external API validates these endpoints. Provide any matching string format to bypass frontend validation.
-

6. Project Structure

Backend Modules

- `controllers/`: Request handling and business logic endpoints.
- `routes/`: Express router mappings.
- `middleware/`: Authentication checks and file upload handling.
- `models/`: Currently delegates to Prisma (via `schema.prisma`).

- `services/`: Encapsulated integrations (e.g., CIBIL mock, Document processing).
- `utils/`: Helpers, formatting, and HMAC signing implementations.
- `tester/` & `__tests__`: Jest automated testing suite and utility scripts.

Frontend Modules

- `app/`: Next.js Route Groups and Pages (Auth, Loan Pages, Main Site, Dashboard).
 - `components/`: UI components (Radix primitives, specialized sections, Form elements).
 - `hooks/`: Specialized state wrappers (`useSignup.ts`, `useLogin.ts`).
 - `lib/`: Standard utilities, Configurations, and the `ApiClient.ts`.
-

7. Operational Notes

- **Testing:** The backend uses Jest and covers primary API routes. Generates HTML testing reports. Run via `npm run test:ci` or `npm run test`.
 - **Database:** Prisma schema changes must be migrated before deploying new core models (`npx prisma migrate deploy`).
 - **Dependencies:** All dependencies are up to date and verified for production. Frontend relies on `@tailwindcss/postcss` and Turbopack for builds. Backend explicitly uses a high memory limit to handle multipart forms.
-

Prepared by: Antigravity **Status:** Verification Completed. Ready for handover.